

```

package nhnis.fw.commons.interceptor;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

import com.fasterxml.jackson.annotation.JsonAutoDetect;
import com.fasterxml.jackson.annotation.PropertyAccessor;
import com.fasterxml.jackson.databind.ObjectMapper;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import nhnis.fw.commons.context.ServiceContextHolder;
import nhnis.fw.commons.dto.header_hdr_nhnis;
import nhnis.fw.commons.exception.NhBaseException;

/**
 * <PRE>
 * A스텝 전/후처리 Interceptor
 * </PRE>
 *
 * @author CSS49257
 * @version 1.0, 2026. 7. 3.
 * @logicalName
 */
@Component
public class ServicePreventionInterceptor implements HandlerInterceptor {

    private Logger LOGGER = LoggerFactory.getLogger(ServicePreventionInterceptor.class);

    private static final String MULTI_PART = "multipart";

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("[ServicePreventionInterceptor] SystemPreProcessor Start!");
        }
    }

```

```

    if (request.getContentType().startsWith(MULTI_PART)) return true;
    if (ServiceContextHolder.getInstance() == null) {
        LOGGER.error("[ServicePreventionInterceptor] Service Context is null...!!");
        return false;
    } else {
        hdr_nhnis header = ServiceContextHolder.getInstance().getHeader();
        String guid = header.getSys_comm().getStd_gbl_id();
        // 1. GUID 체크
        if (guid == null) {
            // generate GUID
        } else {
            LOGGER.info("[ServicePreventionInterceptor] GUID: {}", guid);
        }
        return true;
    }
    // 2. Pre ImageLog
    // 3. 거래제어
}

@Override
public void postHandle(HttpServletRequest request, HttpServletResponse response, Object handler,
    ModelAndView modelAndView) throws Exception {
    if (LOGGER.isInfoEnabled()) {
        LOGGER.info("[ServicePreventionInterceptor] SystemPostProcessor");
    }
    if (request.getContentType().startsWith(MULTI_PART)) {
        // 파일 업로드
    } else {
        // 파일 다운로드
    }
    // Post ImageLog
    HandlerInterceptor.super.postHandle(request, response, handler, modelAndView);
}

```

```

    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object
        handler, Exception ex) throws Exception {
        if (ex != null) {
            LOGGER.error("[ServicePreventionInterceptor] SystemErrorProcessor");
            // Exception ImageLog

            ServiceContextHolder.removeInstance();
            throw ex;
        }
        HandlerInterceptor.super.afterCompletion(request, response, handler, (NhbaseException)ex);
    }

    public <T> T convertMapToDto(Object input, Class<T> type) {
        ObjectMapper mapper = new ObjectMapper();
        mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY);
        return (T) mapper.convertValue(input, type);
    }
}

/*****
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup의 사전 승인 없이
 * 본 내용의 전부 또는 일부를 복사, 배포, 사용 등을 금합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 품질과 성능을 보장하지 않습니다.
 *
 * If you modify this source without Nonghyup's approval, Nonghyup does
 * not guarantee the quality and performance of source.
 *****/
package nhnis.fw.commons.jwt;

```

```

import java.nio.charset.StandardCharsets;

import javax.crypto.SecretKey;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.Jwt;
import io.jsonwebtoken.Keys;
import io.jsonwebtoken.security.Keys;

import jakarta.annotation.PostConstruct;

/**
 * <PRE>
 * <PRE>
 *
 * </PRE>
 *
 * @author CSS49257
 * @version 1.0, 2026. 7. 3.
 * @logicalName
 */
@Component
public class JwtProvider {

    @Value("${jwt.secret}")
    private String secret;

    private SecretKey key;

    @PostConstruct
    public void init() {
        key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public boolean validate(String token) {
        try {
            Jwts.parser()
                .verifyWith(key)
                .build()

```



```

import java.text.MessageFormat;
import java.util.Map;

package nhnis.fw.commons.message;

/*****
 * 2026. 7. 21. 1.0 중립등 최초작성
 *****/
* 립자 버전 작성자 변경사항
*****/
* 이력 사항
*****/
/*****
 * If you modify this source without Nonghyup's approval, Nonghyup does
 * not guarantee the quality and performance of source.
 *****/
*
* Copyright 2026 by Nonghyup. All rights reserved. Nonghyup 의 사전 승인 없이
* 본 내용의 전부 또는 일부에 대한 복사, 배포, 사용을 금합니다. Nonghyup의 사전 승인
* 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 품질과 성능을 보장하지 않습니다.
*****/
}

}

        return "ACCESS".equals(type);
    }

    public boolean isAccessToken(String token) {
        String type = Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload()
            .getSubject();
    }

    public String getSoid(String token) {
        return Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload()
            .getSubject();
    }

    }

    }

    } catch (Exception e) {
        return false;
    }

    .parseSignedClaims(token);

```

